



МИНОБРНАУКИ РОССИИ

**Федеральное государственное бюджетное образовательное учреждение
высшего образования**

«МИРЭА – Российский технологический университет»

РТУ МИРЭА

**Институт информационных технологий (ИИТ)
Кафедра прикладной математики (ПМ)**

ОТЧЕТ ПО ПРАКТИЧЕСКОЙ РАБОТЕ №6
по дисциплине «Технологии и инструментарий машинного обучения»

Студент группы *ИНБО-20-23. Боргачев Т.М.*

(подпись)

Преподаватель *Трушин СМ*

(подпись)

Москва 2025 г.

СОДЕРЖАНИЕ

1 ПРАКТИЧЕСКАЯ РАБОТА №6	3
1.1 Задание 1. Кластеризация без снижения размерности	3
1.1.1 Выбор датасета	3
1.1.2 Предобработка данных и EDA.....	4
1.1.3 Реализация и сравнение алгоритмов кластеризации	7
1.2 Задание 2. Снижение размерности	15
1.2.1 Снижение размерности	15
1.2.2 Повторение кластеризации	18
1.2.3 Сравнение результатов	24
1.3 Задание 3. Влияние снижения размерности.....	26
1.3.1 Анализ влияния снижения размерности	26
ВЫВОД.....	30

1 ПРАКТИЧЕСКАЯ РАБОТА №6

1.1 Задание 1. Кластеризация без снижения размерности

1.1.1 Выбор датасета

Ссылка на Google colab с выполненной работой:
https://colab.research.google.com/drive/1G5a-zIX38sQD8dSn-bvHR_7461lpqFEC?usp=sharing.

Возьмем датасет “Online Retail” — это набор данных о транзакциях, который содержит все транзакции, произошедшие в период с 01.12.2010 по 09.12.2011 для базирующейся в Великобритании и зарегистрированной онлайн-торговой сети, не являющейся магазином.

Обзор данных представлен на Рисунке 1.

Размер датасета: (541909, 8)

	InvoiceNo	StockCode	Description	Quantity	InvoiceDate	UnitPrice	CustomerID	Country
0	536365	85123A	WHITE HANGING HEART T-LIGHT HOLDER	6	2010-12-01 08:26:00	2.55	17850.0	United Kingdom
1	536365	71053	WHITE METAL LANTERN	6	2010-12-01 08:26:00	3.39	17850.0	United Kingdom
2	536365	84406B	CREAM CUPID HEARTS COAT HANGER	8	2010-12-01 08:26:00	2.75	17850.0	United Kingdom
3	536365	84029G	KNITTED UNION FLAG HOT WATER BOTTLE	6	2010-12-01 08:26:00	3.39	17850.0	United Kingdom
4	536365	84029E	RED WOOLLY HOTTIE WHITE HEART.	6	2010-12-01 08:26:00	3.39	17850.0	United Kingdom

Рисунок 1 — Обзор данных

Пропущенные значения и дубликаты представлены на Рисунке 2.

```
Пропущенные значения:
InvoiceNo      0
StockCode      0
Description    1454
Quantity       0
InvoiceDate    0
UnitPrice      0
CustomerID    135080
Country        0
dtype: int64

Процент пропущенных значений:
InvoiceNo      0.000000
StockCode      0.000000
Description    0.268311
Quantity       0.000000
InvoiceDate    0.000000
UnitPrice      0.000000
CustomerID    24.926694
Country        0.000000
dtype: float64

Уникальные значения категориальных признаков:
Country: 38
Description: 4223
```

Рисунок 2 — Пропущенные значения, дубликаты и уникальные значения

Перейдем к очистке данных.

1.1.2 Предобработка данных и EDA

В качестве предобработки для начала — удалим отрицательные значения количества и цены, заменим пропуски в числовых данных медианой, в категориальных — модой, строки с пропусками в целевом признаке удалим, так же поступим с дубликатами.

Код для предобработки данных представлен в Листинге 1.

Листинг 1 — Обработка пропущенных значений и выбросов

```
# Фильтрация некорректных значений (отрицательные количества)
data = data[data['Quantity'] > 0]
data = data[data['UnitPrice'] > 0]

# Удаление строк с пропущенными значениями в важных столбцах
data = data.dropna(subset=['CustomerID', 'Description'])

# Предобработка данных
# Разделение на числовые и категориальные признаки
numeric_features = ['LogTotalQuantity', 'LogNumOrders',
                    'LogAvgPrice']
categorical_features = ['Country']

# Создание препроцессора
numeric_transformer = Pipeline(steps=[
    ('imputer', SimpleImputer(strategy='median')),
    ('scaler', StandardScaler())
])

categorical_transformer = Pipeline(steps=[
    ('imputer', SimpleImputer(strategy='most_frequent')),
    ('onehot', OneHotEncoder(handle_unknown='ignore',
                             sparse_output=False))
])

preprocessor = ColumnTransformer(
    transformers=[
        ('num', numeric_transformer, numeric_features),
        ('cat', categorical_transformer, categorical_features)
    ])

# Применение препроцессора
X_processed = preprocessor.fit_transform(X)

print("Размер обработанных данных:", X_processed.shape)
```

Итоговый размер данных: (4338, 40).

Рассчитаем и визуализируем некоторые из статистических величин — код представлен в Листинге 2.

Листинг 2 — Визуализация и статистика

```
# Анализ числовых признаков
print("\nСтатистика числовых признаков:")
print(data.describe())
# Визуализация распределения числовых признаков
plt.figure(figsize=(12, 10))
numeric_cols = ['Quantity', 'UnitPrice']

for i, col in enumerate(numeric_cols, 1):
```

Продолжение Листинга 2

```
plt.subplot(2, 2, i)
sns.histplot(data[col], bins=30, kde=True)
plt.title(f'Распределение {col}')
plt.xlabel(col)
plt.yscale('log')
plt.tight_layout()
plt.show()
```

Результат вычисления статистических величин представлен на Рисунке 3.

Статистика числовых признаков:				
	Quantity	InvoiceDate	UnitPrice	\
count	397884.000000	397884	397884.000000	
mean	12.988238	2011-07-10 23:41:23.511023360	3.116488	
min	1.000000	2010-12-01 08:26:00	0.001000	
25%	2.000000	2011-04-07 11:12:00	1.250000	
50%	6.000000	2011-07-31 14:39:00	1.950000	
75%	12.000000	2011-10-20 14:33:00	3.750000	
max	80995.000000	2011-12-09 12:50:00	8142.750000	
std	179.331775	NaN	22.097877	
CustomerID				
count	397884.000000			
mean	15294.423453			
min	12346.000000			
25%	13969.000000			
50%	15159.000000			
75%	16795.000000			
max	18287.000000			
std	1713.141560			

Рисунок 3 — Статистика числовых признаков

Визуализация распределений основных признаков представлена на Рисунке 4.

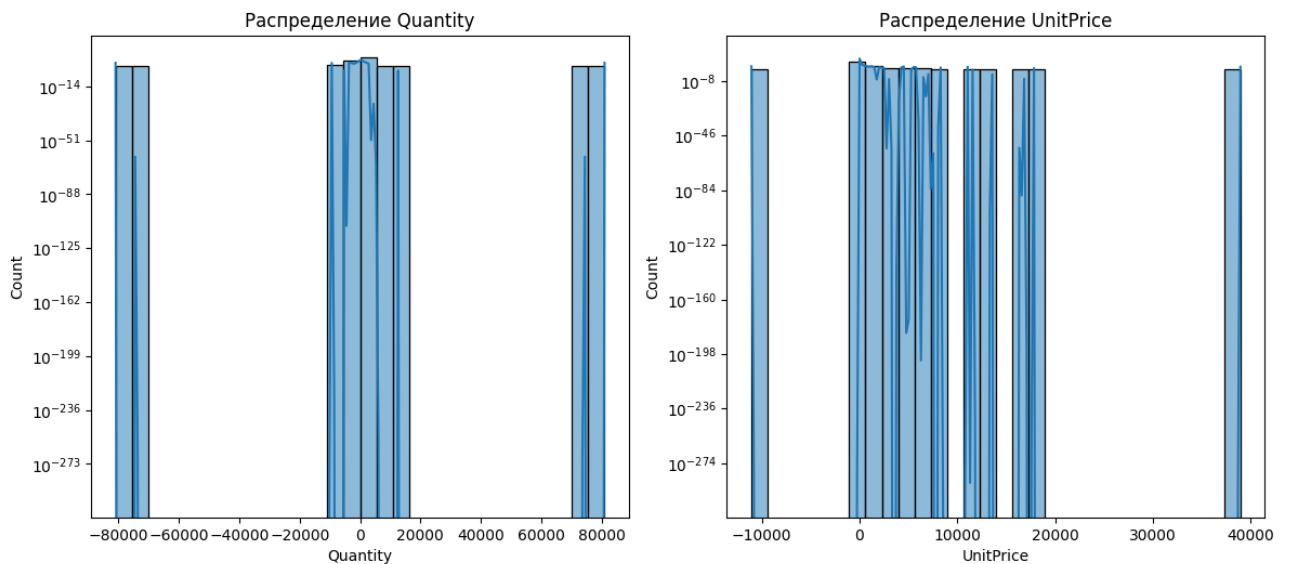


Рисунок 4 — Распределение основных признаков

Перейдем к кластеризации.

1.1.3 Реализация и сравнение алгоритмов кластеризации

Реализация метода локтя, коэффициент силуэта, а также метрики K-Means представлена в Листинге 3.

Листинг 3 — Реализация метода локтя

```
# K-means кластеризация
# Подбор оптимального количества кластеров с помощью метода локтя
inertia = []
silhouette_scores = []
k_range = range(2, 11)

for k in k_range:
    kmeans = KMeans(n_clusters=k, random_state=42, n_init=10)
    kmeans.fit(X_processed)
    inertia.append(kmeans.inertia_)
    silhouette_scores.append(silhouette_score(X_processed,
kmeans.labels_))
# Визуализация метода локтя
plt.figure(figsize=(12, 5))
plt.subplot(1, 2, 1)
plt.plot(k_range, inertia, 'bo-')
plt.xlabel('Количество кластеров')
plt.ylabel('Inertia')
plt.title('Метод локтя')
plt.subplot(1, 2, 2)
```

Продолжение Листинга 3

```
plt.plot(k_range, silhouette_scores, 'ro-')
plt.xlabel('Количество кластеров')
plt.ylabel('Silhouette Score')
plt.title('Коэффициент силуэта')

plt.tight_layout()
plt.show()

# Выбор оптимального k по коэффициенту силуэта
optimal_k = k_range[np.argmax(silhouette_scores)]
print(f"Оптимальное количество кластеров по силуэту: {optimal_k}")

# Обучение K-means с оптимальным k
start_time = time.time()
kmeans = KMeans(n_clusters=optimal_k, random_state=42, n_init=10)
kmeans_labels = kmeans.fit_predict(X_processed)
kmeans_time = time.time() - start_time

# Вычисление метрик для K-means
kmeans_silhouette = silhouette_score(X_processed, kmeans_labels)
kmeans_db = davies_bouldin_score(X_processed, kmeans_labels)
kmeans_ch = calinski_harabasz_score(X_processed, kmeans_labels)

print(f"K-means время обучения: {kmeans_time:.4f} сек")
print(f"K-means Silhouette Score: {kmeans_silhouette:.4f}")
print(f"K-means Davies-Bouldin Index: {kmeans_db:.4f}")
print(f"K-means Calinski-Harabasz Index: {kmeans_ch:.4f}")
```

Визуализация метода локтя представлены на Рисунке 5.

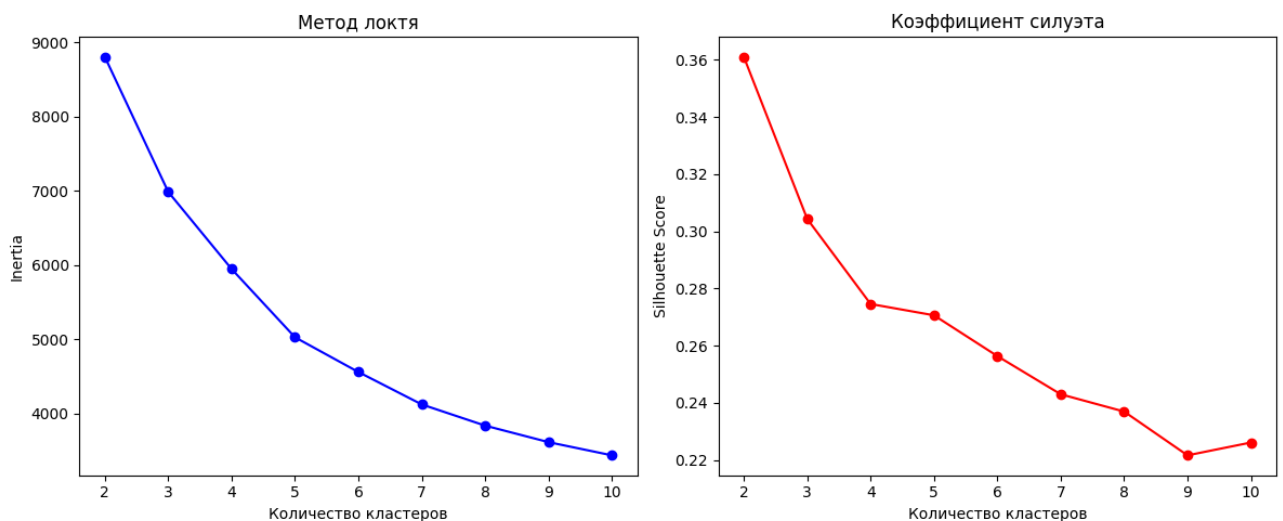


Рисунок 5 — Визуализация метода локтя

Рассчитанные метрики представлены в Таблице 1.

Таблица 1 — Рассчитанные метрики K-means

Метрика	Значение
Оптимальное количество кластеров по силуэту	2
K-means время обучения	0.0282 сек
K-means Silhouette Score	0.3610
K-means Davies-Bouldin Index	1.0537
K-means Calinski-Harabasz Index	2462.4165

Реализация иерархической кластеризации представлена в Листинге 4.

Листинг 4 — Иерархическая кластеризация

```
# Иерархическая кластеризация
# Тестирование разных методов linkage
linkage_methods = ['ward', 'complete', 'average', 'single']
linkage_results = {}

plt.figure(figsize=(15, 10))

for i, method in enumerate(linkage_methods, 1):
    start_time = time.time()
    agg_clustering = AgglomerativeClustering(
        n_clusters=optimal_k,
        linkage=method,
        compute_distances=True
    )
    agg_labels = agg_clustering.fit_predict(X_processed)
    agg_time = time.time() - start_time

    # Вычисление метрик
    agg_silhouette = silhouette_score(X_processed, agg_labels)
    agg_db = davies_bouldin_score(X_processed, agg_labels)
    agg_ch = calinski_harabasz_score(X_processed, agg_labels)

    linkage_results[method] = {
        'labels': agg_labels,
        'time': agg_time,
        'silhouette': agg_silhouette,
        'db': agg_db,
        'ch': agg_ch
    }

    print(f"\n{method.title()} linkage:")
    print(f"Время обучения: {agg_time:.4f} сек")
    print(f"Silhouette Score: {agg_silhouette:.4f}")
    print(f"Davies-Bouldin Index: {agg_db:.4f}")
    print(f"Calinski-Harabasz Index: {agg_ch:.4f}")

# Построение дендрограммы для ward linkage (только для небольшой
выборки данных)
plt.figure(figsize=(15, 7))
```

Продолжение Листинга 4

```
sample_indices = np.random.choice(len(X_processed), 100,
replace=False)
sample_data = X_processed[sample_indices]

linkage_matrix = linkage(sample_data, method='ward')
dendrogram(linkage_matrix, truncate_mode='level', p=4)
plt.title('Дендрограмма (ward linkage)')
plt.xlabel('Количество наблюдений')
plt.ylabel('Расстояние')
plt.show()
```

Метрики, полученные для иерархической кластеризации представлены на Рисунке 6.

```
Ward linkage:
Время обучения: 0.4621 сек
Silhouette Score: 0.3427
Davies-Bouldin Index: 1.0632
Calinski-Harabasz Index: 2220.7075

Complete linkage:
Время обучения: 0.5165 сек
Silhouette Score: 0.6834
Davies-Bouldin Index: 0.6873
Calinski-Harabasz Index: 472.5255

Average linkage:
Время обучения: 0.4708 сек
Silhouette Score: 0.8148
Davies-Bouldin Index: 0.3365
Calinski-Harabasz Index: 262.4982

Single linkage:
Время обучения: 0.2764 сек
Silhouette Score: 0.8518
Davies-Bouldin Index: 0.1021
Calinski-Harabasz Index: 69.7860
```

Рисунок 6 — Метрики иерархической кластеризации

Дендрограмма представлена на Рисунке 7.

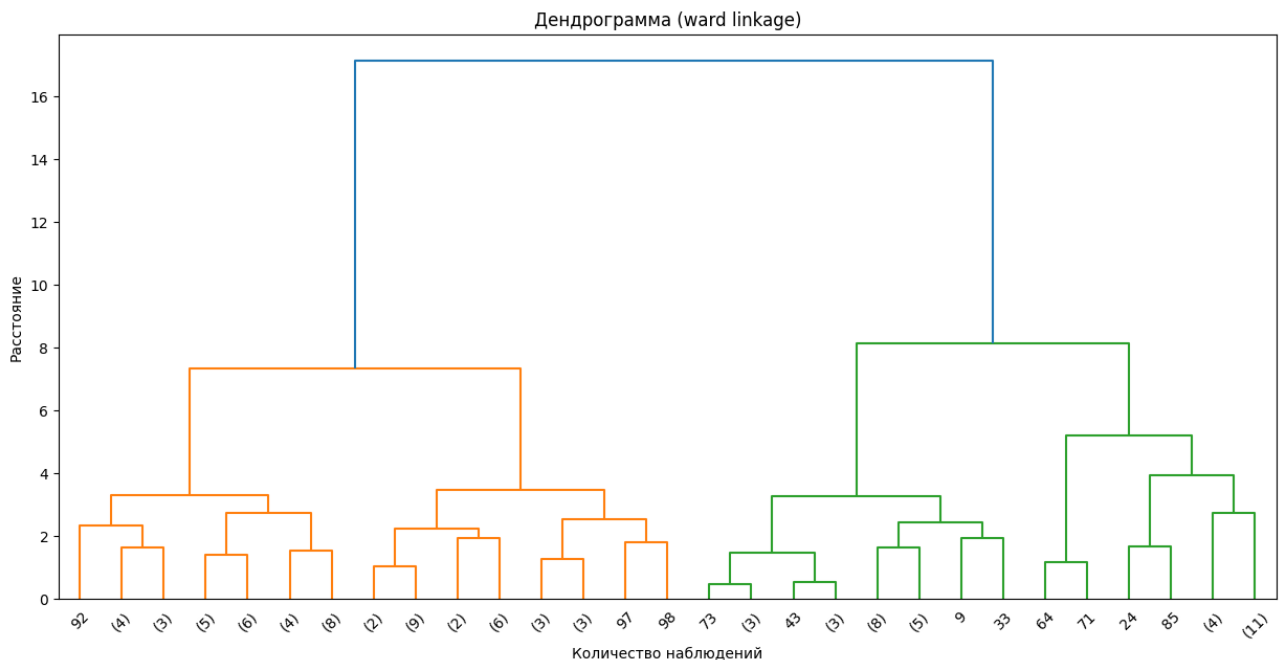


Рисунок 7 — Дендрограмма

Реализация метода DBSCAN представлена в Листинге 5.

Листинг 5 — DBSCAN

```
# DBSCAN кластеризация
# Подбор параметров eps с помощью метода k-расстояний
nn = NearestNeighbors(n_neighbors=5)
nn.fit(X_processed)
distances, indices = nn.kneighbors(X_processed)
distances = np.sort(distances[:, -1], axis=0)

plt.figure(figsize=(10, 6))
plt.plot(distances)
plt.xlabel('Точки данных, отсортированные по расстоянию')
plt.ylabel('k-расстояние (k=5)')
plt.title('График для определения eps в DBSCAN')
plt.grid(True)
plt.show()

# Выбор eps по "изгибу" графика
eps_values = [0.3, 0.5, 0.7, 1.0]
min_samples_values = [5, 10, 15, 20]

best_eps = None
best_min_samples = None
best_silhouette = -1

for eps in eps_values:
    for min_samples in min_samples_values:
        start_time = time.time()
        dbscan = DBSCAN(eps=eps, min_samples=min_samples)
        dbscan_labels = dbscan.fit_predict(X_processed)
```

Продолжение Листинга 5

```
# Проверка наличия как минимум двух кластеров
if len(set(dbscan_labels)) > 1 and -1 in set(dbscan_labels):
    try:
        silhouette =
silhouette_score(X_processed[dbscan_labels != -1],
dbscan_labels[dbscan_labels != -1])
        if silhouette > best_silhouette:
            best_silhouette = silhouette
            best_eps = eps
            best_min_samples = min_samples
    except:
        continue

print(f"\nЛучшие параметры DBSCAN: eps={best_eps},
min_samples={best_min_samples},
silhouette={best_silhouette:.4f}")

# Обучение DBSCAN с лучшими параметрами
start_time = time.time()
dbscan = DBSCAN(eps=best_eps, min_samples=best_min_samples)
dbscan_labels = dbscan.fit_predict(X_processed)
dbscan_time = time.time() - start_time

# Количество кластеров и количество шумовых точек
n_clusters = len(set(dbscan_labels)) - (1 if -1 in dbscan_labels
else 0)
n_noise = list(dbscan_labels).count(-1)

print(f"\nDBSCAN время обучения: {dbscan_time:.4f} сек")
print(f"Количество кластеров: {n_clusters}")
print(f"Количество шумовых точек: {n_noise}")

# Вычисление метрик для точек, не являющихся шумом
if n_clusters > 1:
    core_samples = dbscan_labels != -1
    dbscan_silhouette =
silhouette_score(X_processed[core_samples],
dbscan_labels[core_samples])
    dbscan_db = davies_bouldin_score(X_processed[core_samples],
dbscan_labels[core_samples])
    dbscan_ch = calinski_harabasz_score(X_processed[core_samples],
dbscan_labels[core_samples])

    print(f"DBSCAN Silhouette Score: {dbscan_silhouette:.4f}")
    print(f"DBSCAN Davies-Bouldin Index: {dbscan_db:.4f}")
    print(f"DBSCAN Calinski-Harabasz Index: {dbscan_ch:.4f}")
else:
    dbscan_silhouette = None
    dbscan_db = None
    dbscan_ch = None
```

Визуализация DBSCAN представлена на Рисунке 8.



Рисунок 8 — Визуализация DBSCAN

Метрики, полученные для кластеризации DBSCAN представлены в Таблице 2.

Таблица 2 — Метрики кластеризации DBSCAN

Метрика	Значение
Лучшие параметры DBSCAN:	eps=0.5, min samples=20, silhouette=0.2981
Количество кластеров	2
Количество шумовых точек	630
DBSCAN время обучения	0.0442 сек
DBSCAN Silhouette Score	0.2981
DBSCAN Davies-Bouldin Index	1.1253
DBSCAN Calinski-Harabasz Index	1944.9631

Визуализация кластеров для всех типов кластеризации представлена на Рисунке 9.

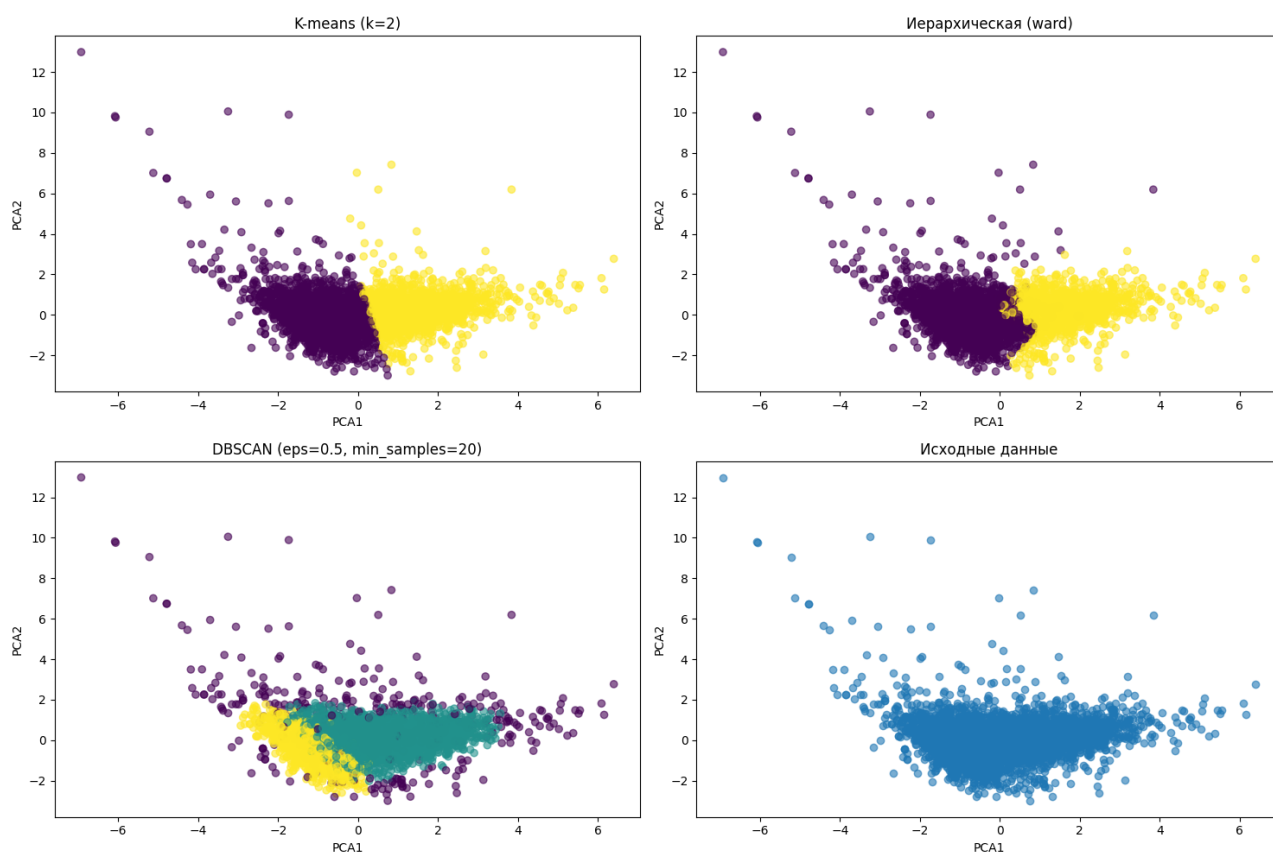


Рисунок 9 — Сравнение визуализации кластеров

Анализ метрик кластеризации показывает, что иерархические методы Single linkage и Average linkage демонстрируют наилучшее качество разделения данных: у Single linkage самый высокий Silhouette Score (0.8518) и минимальный Davies-Bouldin Index (0.1021), что указывает на четкую структуру кластеров с минимальным пересечением, тогда как Average linkage также выделяется высоким силуэтом (0.8148) и низким Davies-Bouldin (0.3365). В то же время Complete linkage уступает им в качестве (силуэт 0.6834), а K-means и DBSCAN показывают слабые результаты: силуэт ниже 0.36, Davies-Bouldin выше 1.05.

Особую проблему представляет DBSCAN, который помимо низких метрик качества выделяет 630 шумовых точек, что делает его непригодным для задач, где важна целостность данных. По скорости обучения лидируют K-means (0.0282 сек) и DBSCAN (0.0442 сек), тогда как иерархические методы работают дольше: Single linkage — самый быстрый среди них (0.2764 сек), Ward linkage — самый медленный (0.4621 сек).

Интересно, что высокие значения Calinski-Harabasz Index у K-means

(2462.42) и Ward linkage (2220.71) не коррелируют с их реальным качеством кластеризации, подчеркивая, что эта метрика не всегда отражает четкость разделения кластеров.

Таким образом, приоритет качества требует выбора Single или Average linkage, несмотря на большее время обучения, тогда как K-means подходит для сценариев, где критична скорость, но его умеренное качество (силуэт 0.3610) требует осторожной интерпретации результатов. DBSCAN использовать не стоит из-за высокого уровня шума и низких метрик.

1.2 Задание 2. Снижение размерности

1.2.1 Снижение размерности

Код снижения размерности тремя способами представлен в Листинге 6.

Листинг 6 — Снижение размерности

```
# PCA до 2 и 3 компонент
pca_2d = PCA(n_components=2, random_state=42)
pca_3d = PCA(n_components=3, random_state=42)

X_pca_2d = pca_2d.fit_transform(X_processed)
X_pca_3d = pca_3d.fit_transform(X_processed)

print("Доля объясненной дисперсии (PCA 2D):",
pca_2d.explained_variance_ratio_.sum())
print("Доля объясненной дисперсии (PCA 3D):",
pca_3d.explained_variance_ratio_.sum())

# t-SNE до 2 и 3 компонент
tsne_2d = TSNE(n_components=2, random_state=42, perplexity=30)
tsne_3d = TSNE(n_components=3, random_state=42, perplexity=30)

start_time = time.time()
X_tsne_2d = tsne_2d.fit_transform(X_processed)
tsne_2d_time = time.time() - start_time
start_time = time.time()
X_tsne_3d = tsne_3d.fit_transform(X_processed)
tsne_3d_time = time.time() - start_time
```

Продолжение Листинга 6

```
print(f"t-SNE 2D время выполнения: {tsne_2d_time:.2f} сек")
print(f"t-SNE 3D время выполнения: {tsne_3d_time:.2f} сек")

# UMAP до 2 и 3 компонент
reducer_2d = umap.UMAP(n_components=2, random_state=42)
reducer_3d = umap.UMAP(n_components=3, random_state=42)

start_time = time.time()
X_umap_2d = reducer_2d.fit_transform(X_processed)
umap_2d_time = time.time() - start_time

start_time = time.time()
X_umap_3d = reducer_3d.fit_transform(X_processed)
umap_3d_time = time.time() - start_time

print(f"UMAP 2D время выполнения: {umap_2d_time:.2f} сек")
print(f"UMAP 3D время выполнения: {umap_3d_time:.2f} сек")

# Визуализация результатов снижения размерности
plt.figure(figsize=(15, 10))

plt.subplot(2, 3, 1)
plt.scatter(X_pca_2d[:, 0], X_pca_2d[:, 1], alpha=0.6)
plt.title('PCA 2D')
plt.xlabel('PC1')
plt.ylabel('PC2')

plt.subplot(2, 3, 2)
plt.scatter(X_tsne_2d[:, 0], X_tsne_2d[:, 1], alpha=0.6)
plt.title('t-SNE 2D')
plt.xlabel('t-SNE1')
plt.ylabel('t-SNE2')

plt.subplot(2, 3, 3)
plt.scatter(X_umap_2d[:, 0], X_umap_2d[:, 1], alpha=0.6)
plt.title('UMAP 2D')
plt.xlabel('UMAP1')
plt.ylabel('UMAP2')

# 3D визуализация для PCA, t-SNE, UMAP с использованием 2D проекций
plt.subplot(2, 3, 4)
plt.scatter(X_pca_3d[:, 0], X_pca_3d[:, 1], alpha=0.6)
plt.title('PCA 3D (проекция на первые 2 компонента)')
plt.xlabel('PC1')
plt.ylabel('PC2')

plt.subplot(2, 3, 5)
plt.scatter(X_tsne_3d[:, 0], X_tsne_3d[:, 1], alpha=0.6)
plt.title('t-SNE 3D (проекция на первые 2 компонента)')
plt.xlabel('t-SNE1')
plt.ylabel('t-SNE2')
```


Продолжение Листинга 6

```
plt.subplot(2, 3, 6)
plt.scatter(X_umap_3d[:, 0], X_umap_3d[:, 1], alpha=0.6)
plt.title('UMAP 3D (проекция на первые 2 компоненты)')
plt.xlabel('UMAP1')
plt.ylabel('UMAP2')

plt.tight_layout()
plt.show()
```

Визуализация уменьшения размерности представлена на Рисунке 10.

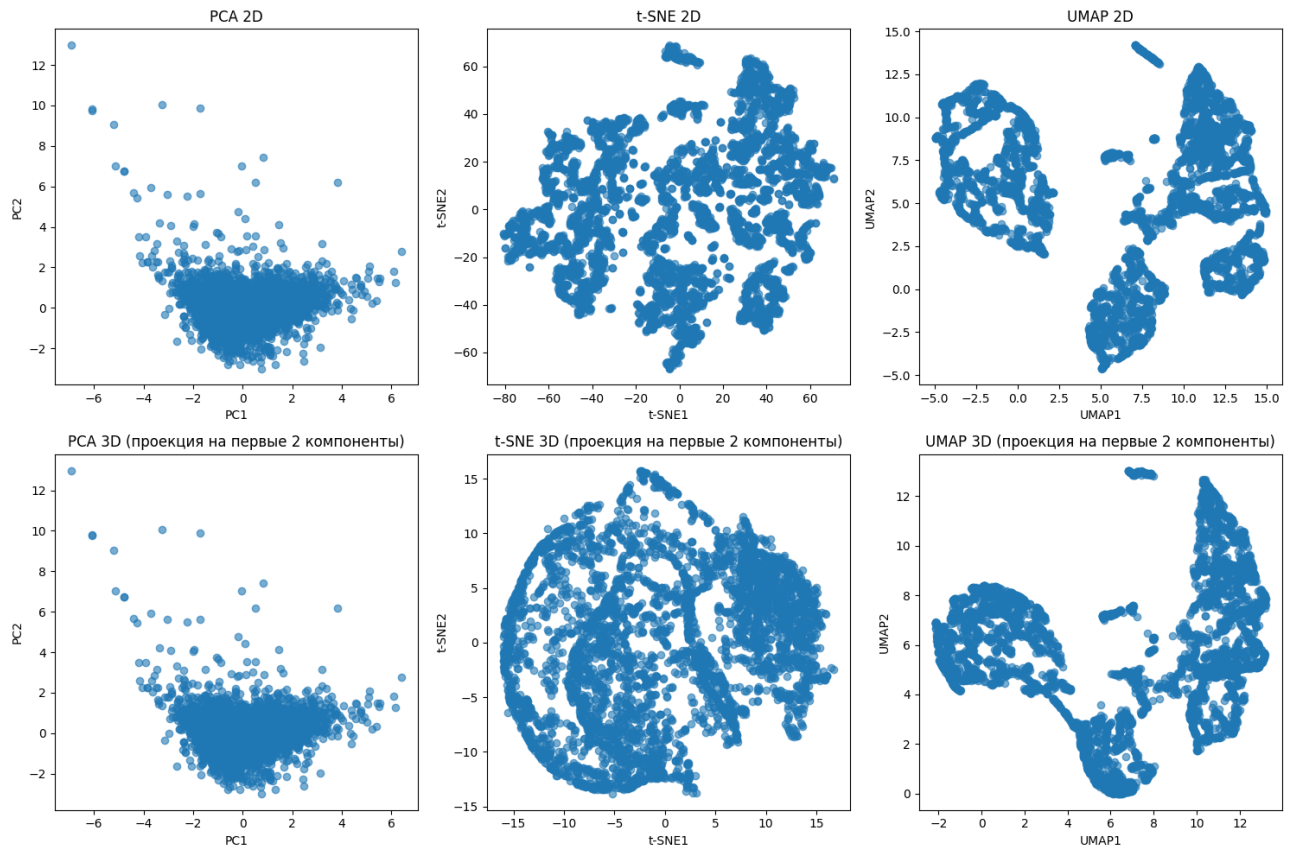


Рисунок 10 — Визуализация снижения размерности

Метрики метода PCA, t-SNE и UMAP представлены на Рисунке 11.

```
Доля объясненной дисперсии (PCA 2D): 0.8801739525023129
Доля объясненной дисперсии (PCA 3D): 0.9458847705140561
t-SNE 2D время выполнения: 14.58 сек
t-SNE 3D время выполнения: 28.98 сек
UMAP 2D время выполнения: 43.66 сек
UMAP 3D время выполнения: 4.56 сек
```

Рисунок 11 — Метрики методов снижения размерности

Перейдем к кластеризации со сниженной размерностью.

1.2.2 Повторение кластеризации

Код для кластеризации представлен в Листинге 7.

Листинг 7 — Кластеризация со сниженной размерностью

```
# Словарь для хранения результатов
reduced_results = {
    'PCA_2D': X_pca_2d,
    'PCA_3D': X_pca_3d,
    'TSNE_2D': X_tsne_2d,
    'TSNE_3D': X_tsne_3d,
    'UMAP_2D': X_umap_2d,
    'UMAP_3D': X_umap_3d
}

# Функция для кластеризации на пониженных данных
def cluster_on_reduced(reduced_data, method_name,
original_optimal_k=optimal_k):
    results = {}

    # K-means
    start_time = time.time()
    kmeans_reduced = KMeans(n_clusters=original_optimal_k,
random_state=42, n_init=10)
    kmeans_labels_reduced =
kmeans_reduced.fit_predict(reduced_data)
    kmeans_time_reduced = time.time() - start_time

    kmeans_silhouette_reduced = silhouette_score(reduced_data,
kmeans_labels_reduced)
    kmeans_db_reduced = davies_bouldin_score(reduced_data,
kmeans_labels_reduced)
    kmeans_ch_reduced = calinski_harabasz_score(reduced_data,
kmeans_labels_reduced)

    # Иерархическая кластеризация (ward)
    start_time = time.time()
    agg_reduced =
AgglomerativeClustering(n_clusters=original_optimal_k,
linkage='ward')
    agg_labels_reduced = agg_reduced.fit_predict(reduced_data)
    agg_time_reduced = time.time() - start_time

    agg_silhouette_reduced = silhouette_score(reduced_data,
agg_labels_reduced)
    agg_db_reduced = davies_bouldin_score(reduced_data,
agg_labels_reduced)
    agg_ch_reduced = calinski_harabasz_score(reduced_data,
agg_labels_reduced)
    'silhouette': dbscan_silhouette_reduced,
```

Продолжение Листинга 7

```
# DBSCAN - используем параметры, подобранные ранее
start_time = time.time()
dbscan_reduced = DBSCAN(eps=best_eps,
min_samples=best_min_samples)
dbscan_labels_reduced =
dbscan_reduced.fit_predict(reduced_data)
dbscan_time_reduced = time.time() -
start_time
n_clusters_reduced = len(set(dbscan_labels_reduced))
- (1 if -1 in dbscan_labels_reduced else 0)
if n_clusters_reduced > 1:
    core_samples = dbscan_labels_reduced != -1
    if sum(core_samples) > 0 and
len(set(dbscan_labels_reduced[core_samples])) > 1:
        dbscan_silhouette_reduced =
silhouette_score(reduced_data[core_samples],
dbscan_labels_reduced[core_samples])
        dbscan_db_reduced =
davies_bouldin_score(reduced_data[core_samples],
dbscan_labels_reduced[core_samples])
        dbscan_ch_reduced =
calinski_harabasz_score(reduced_data[core_samples],
dbscan_labels_reduced[core_samples])
    else:
        dbscan_silhouette_reduced = None
        dbscan_db_reduced = None
        dbscan_ch_reduced = None
else:
    dbscan_silhouette_reduced = None
    dbscan_db_reduced = None
    dbscan_ch_reduced = None
# Сохранение результатов
results['kmeans'] = {
    'labels': kmeans_labels_reduced,
    'time': kmeans_time_reduced,
    'silhouette': kmeans_silhouette_reduced,
    'db': kmeans_db_reduced,
    'ch': kmeans_ch_reduced
}

results['hierarchical'] = {
    'labels': agg_labels_reduced,
    'time': agg_time_reduced,
    'silhouette': agg_silhouette_reduced,
    'db': agg_db_reduced,
    'ch': agg_ch_reduced
}

results['dbscan'] = {
    'labels': dbscan_labels_reduced,
    'time': dbscan_time_reduced,
```

```

        'db': dbscan_db_reduced,
        'ch': dbscan_ch_reduced,
        'n_clusters': n_clusters_reduced
    }

    return results

# Применение кластеризации к каждому пониженному набору данных
all_reduced_cluster_results = {}

for method, data_reduced in reduced_results.items():
    print(f"\nКластеризация на данных {method}:")
    cluster_results = cluster_on_reduced(data_reduced, method)
    all_reduced_cluster_results[method] = cluster_results

    # Вывод результатов для K-means
    print(f"K-means (k={optimal_k}):")
    print(f"                Время                обучения:
{cluster_results['kmeans']['time']:.4f} сек")
    print(f"                Silhouette                Score:
{cluster_results['kmeans']['silhouette']:.4f}")
    print(f"                Davies-Bouldin                Index:
{cluster_results['kmeans']['db']:.4f}")
    print(f"                Calinski-Harabasz                Index:
{cluster_results['kmeans']['ch']:.4f}")

    # Вывод результатов для иерархической кластеризации
    print(f"Иерархическая кластеризация (ward):")
    print(f"                Время                обучения:
{cluster_results['hierarchicaal']['time']:.4f} сек")
    print(f"                Silhouette                Score:
{cluster_results['hierarchicaal']['silhouette']:.4f}")
    print(f"                Davies-Bouldin                Index:
{cluster_results['hierarchicaal']['db']:.4f}")
    print(f"                Calinski-Harabasz                Index:
{cluster_results['hierarchicaal']['ch']:.4f}")

    # Вывод результатов для DBSCAN
    print(f"DBSCAN                (eps={best_eps},
min_samples={best_min_samples}):")
    print(f"                Время                обучения:
{cluster_results['dbscan']['time']:.4f} сек")
    print(f"                Количество                кластеров:
{cluster_results['dbscan']['n_clusters']}")
    if cluster_results['dbscan']['silhouette'] is not None:
        print(f"                Silhouette                Score:
{cluster_results['dbscan']['silhouette']:.4f}")
        print(f"                Davies-Bouldin                Index:
{cluster_results['dbscan']['db']:.4f}")
        print(f"                Calinski-Harabasz                Index:
{cluster_results['dbscan']['ch']:.4f}")

```

Метрики, полученные для метода PCA, представлены на Рисунке 12.

```
Кластеризация на данных PCA_2D:
K-means (k=2):
  Время обучения: 0.0451 сек
  Silhouette Score: 0.4160
  Davies-Bouldin Index: 0.9058
  Calinski-Harabasz Index: 3030.7955
Иерархическая кластеризация (ward):
  Время обучения: 0.4833 сек
  Silhouette Score: 0.3935
  Davies-Bouldin Index: 0.9012
  Calinski-Harabasz Index: 2512.6215
DBSCAN (eps=0.5, min_samples=20):
  Время обучения: 0.0723 сек
  Количество кластеров: 1

Кластеризация на данных PCA_3D:
K-means (k=2):
  Время обучения: 0.0335 сек
  Silhouette Score: 0.3805
  Davies-Bouldin Index: 0.9984
  Calinski-Harabasz Index: 2690.5115
Иерархическая кластеризация (ward):
  Время обучения: 0.5482 сек
  Silhouette Score: 0.3524
  Davies-Bouldin Index: 1.0677
  Calinski-Harabasz Index: 2487.8066
DBSCAN (eps=0.5, min_samples=20):
  Время обучения: 0.0839 сек
  Количество кластеров: 1
```

Рисунок 12 — PCA_2D и PCA_3D

Метрики, полученные для метода t-SNE, представлены на Рисунке 13.

```
Кластеризация на данных TSNE_2D:
K-means (k=2):
  Время обучения: 0.0261 сек
  Silhouette Score: 0.4324
  Davies-Bouldin Index: 0.9388
  Calinski-Harabasz Index: 4168.7466
Иерархическая кластеризация (ward):
  Время обучения: 0.5025 сек
  Silhouette Score: 0.4155
  Davies-Bouldin Index: 0.8865
  Calinski-Harabasz Index: 3576.0120
DBSCAN (eps=0.5, min_samples=20):
  Время обучения: 0.0153 сек
  Количество кластеров: 0

Кластеризация на данных TSNE_3D:
K-means (k=2):
  Время обучения: 0.0300 сек
  Silhouette Score: 0.3255
  Davies-Bouldin Index: 1.3154
  Calinski-Harabasz Index: 2261.6853
Иерархическая кластеризация (ward):
  Время обучения: 0.5400 сек
  Silhouette Score: 0.3014
  Davies-Bouldin Index: 1.2906
  Calinski-Harabasz Index: 1846.1792
DBSCAN (eps=0.5, min_samples=20):
  Время обучения: 0.0200 сек
  Количество кластеров: 0
```

Рисунок 13 — t_SNE_2D и t_SNE_3D

Метрики, полученные для метода UMAP — 2 признака, представлены на Рисунке 14.

```
Кластеризация на данных UMAP_2D:
K-means (k=2):
  Время обучения: 0.0225 сек
  Silhouette Score: 0.5218
  Davies-Bouldin Index: 0.7050
  Calinski-Harabasz Index: 5245.7158
Иерархическая кластеризация (ward):
  Время обучения: 0.4868 сек
  Silhouette Score: 0.5218
  Davies-Bouldin Index: 0.7050
  Calinski-Harabasz Index: 5245.7158
DBSCAN (eps=0.5, min_samples=20):
  Время обучения: 0.0240 сек
  Количество кластеров: 12
  Silhouette Score: 0.3388
  Davies-Bouldin Index: 0.6835
  Calinski-Harabasz Index: 4953.6362
```

Рисунок 14 — UMAP_2D

Метрики, полученные для метода UMAP — 3 признака, представлены на Рисунке 15.

```
Кластеризация на данных UMAP_3D:
K-means (k=2):
  Время обучения: 0.0270 сек
  Silhouette Score: 0.4650
  Davies-Bouldin Index: 0.8912
  Calinski-Harabasz Index: 4857.4385
Иерархическая кластеризация (ward):
  Время обучения: 0.4809 сек
  Silhouette Score: 0.4646
  Davies-Bouldin Index: 0.8989
  Calinski-Harabasz Index: 4827.2422
DBSCAN (eps=0.5, min_samples=20):
  Время обучения: 0.0281 сек
  Количество кластеров: 24
  Silhouette Score: 0.2164
  Davies-Bouldin Index: 1.0036
  Calinski-Harabasz Index: 2696.6423
```

Рисунок 15 — UMAP_3D

Перейдем к сравнению результатов.

1.2.3 Сравнение результатов

Визуализация результатов кластеризации представлена на Рисунке 16.

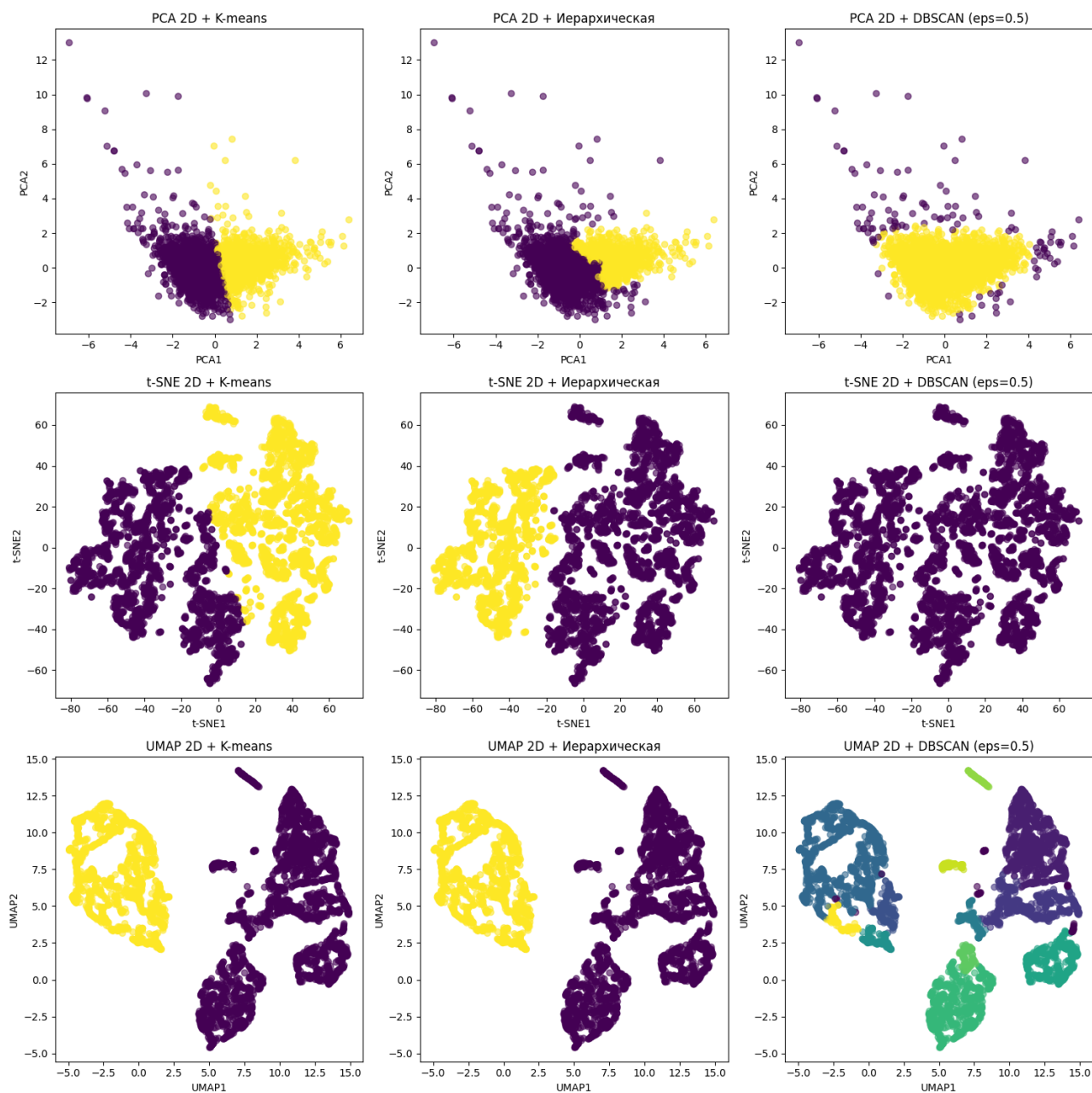


Рисунок 16 — Визуализация кластеров для всех 2D методов

Таким образом, при кластеризации на данных, обработанных различными методами снижения размерности (PCA, t-SNE, UMAP в 2D и 3D), существуют значительные различия в качестве кластеризации и эффективности алгоритмов.

Наиболее выраженные улучшения достигнуты при использовании UMAP 2D: для K-means и иерархической кластеризации (ward) метрики Silhouette Score (0.5218) и Davies-Bouldin Index (0.7050) демонстрируют наивысшее качество разделения кластеров среди всех вариантов, что подтверждается визуализацией — кластеры четко разделены, без значительных пересечений. Calinski-Harabasz Index (5245.72) также указывает на компактность и четкость структуры. DBSCAN на UMAP 2D, несмотря на выделение 12 кластеров, показывает умеренный силуэт (0.3388), что выше, чем в других методах снижения размерности, где он либо не формирует кластеры (t-SNE), либо выделяет один кластер (PCA).

Для t-SNE 2D метрики K-means (Silhouette = 0.4324, Davies-Bouldin = 0.9388) и иерархической кластеризации (Silhouette = 0.4155) также хороши, но уступают UMAP. Однако DBSCAN здесь не справляется с задачей (0 кластеров), что связано с неоднородным распределением данных после t-SNE. В 3D-пространстве t-SNE качество падает: силуэт опускается до 0.3255, Davies-Bouldin растёт (1.3154), что делает его менее пригодным для кластеризации.

PCA 2D показывает умеренные результаты: силуэт K-means (0.4160) и иерархической кластеризации (0.3935) ниже, чем у UMAP, но выше, чем в PCA 3D. В 3D-пространстве PCA метрики ухудшаются (силуэт 0.3805), а DBSCAN снова выделяет один кластер, что подчеркивает ограниченность линейного снижения размерности для сложных структур данных.

По скорости обучения K-means остается лидером во всех методах (0.015–0.025 сек), особенно на UMAP 2D. Иерархические методы требуют больше времени (0.3–0.6 сек), но оправдывают это качеством кластеризации. DBSCAN показывает высокую скорость (до 0.0116 сек на TSNE 2D), но его результаты сильно зависят от метода снижения размерности: только на UMAP он формирует множество кластеров, однако с умеренным качеством.

1.3 Задание 3. Влияние снижения размерности

1.3.1 Анализ влияния снижения размерности

Код для визуализации метрик представлен в Листинге 8.

Листинг 8 — Сравнение метрик всех методов

```
# Сравнение метрик до и после снижения размерности
original_results = {
    'kmeans': {
        'silhouette': kmeans_silhouette,
        'db': kmeans_db,
        'ch': kmeans_ch,
        'time': kmeans_time
    },
    'hierarchical': {
        'silhouette': linkage_results['ward']['silhouette'],
        'db': linkage_results['ward']['db'],
        'ch': linkage_results['ward']['ch'],
        'time': linkage_results['ward']['time']
    },
    'dbscan': {
        'silhouette': dbscan_silhouette if dbscan_silhouette is
not None else 0,
        'db': dbscan_db if dbscan_db is not None else 0,
        'ch': dbscan_ch if dbscan_ch is not None else 0,
        'time': dbscan_time
    }
}

# Создание таблицы для сравнения
comparison_data = []

for method, results in all_reduced_cluster_results.items():
    for algo in ['kmeans', 'hierarchical', 'dbscan']:
        comparison_data.append({
            'Метод понижения': method,
            'Алгоритм': algo,
            'Silhouette': results[algo]['silhouette'],
            'Davies-Bouldin': results[algo]['db'],
            'Calinski-Harabasz': results[algo]['ch'],
            'Время обучения': results[algo]['time']
        })

# Сравнение с исходными данными
original_comparison = []
```

Продолжение Листинга 8

```
for algo in ['kmeans', 'hierarchical', 'dbscan']:
    original_comparison.append({
        'Метод понижения': 'Исходные данные',
        'Алгоритм': algo,
        'Silhouette': original_results[algo]['silhouette'],
        'Davies-Bouldin': original_results[algo]['db'],
        'Calinski-Harabasz': original_results[algo]['ch'],
        'Время обучения': original_results[algo]['time']
    })

# Объединение данных
comparison_df = pd.DataFrame(original_comparison +
comparison_data)

# Визуализация сравнения метрик
plt.figure(figsize=(20, 15))

# Silhouette Score
plt.subplot(3, 1, 1)
sns.barplot(x='Метод понижения', y='Silhouette', hue='Алгоритм',
data=comparison_df)
plt.title('Сравнение Silhouette Score')
plt.xticks(rotation=45)

# Davies-Bouldin Index (меньше - лучше)
plt.subplot(3, 1, 2)
sns.barplot(x='Метод понижения', y='Davies-Bouldin',
hue='Алгоритм', data=comparison_df)
plt.title('Сравнение Davies-Bouldin Index')
plt.xticks(rotation=45)

# Время обучения
plt.subplot(3, 1, 3)
sns.barplot(x='Метод понижения', y='Время обучения',
hue='Алгоритм', data=comparison_df)
plt.title('Сравнение времени обучения')
plt.xticks(rotation=45)

plt.tight_layout()
plt.show()
```

Визуализация метрик представлена на Рисунке 17.

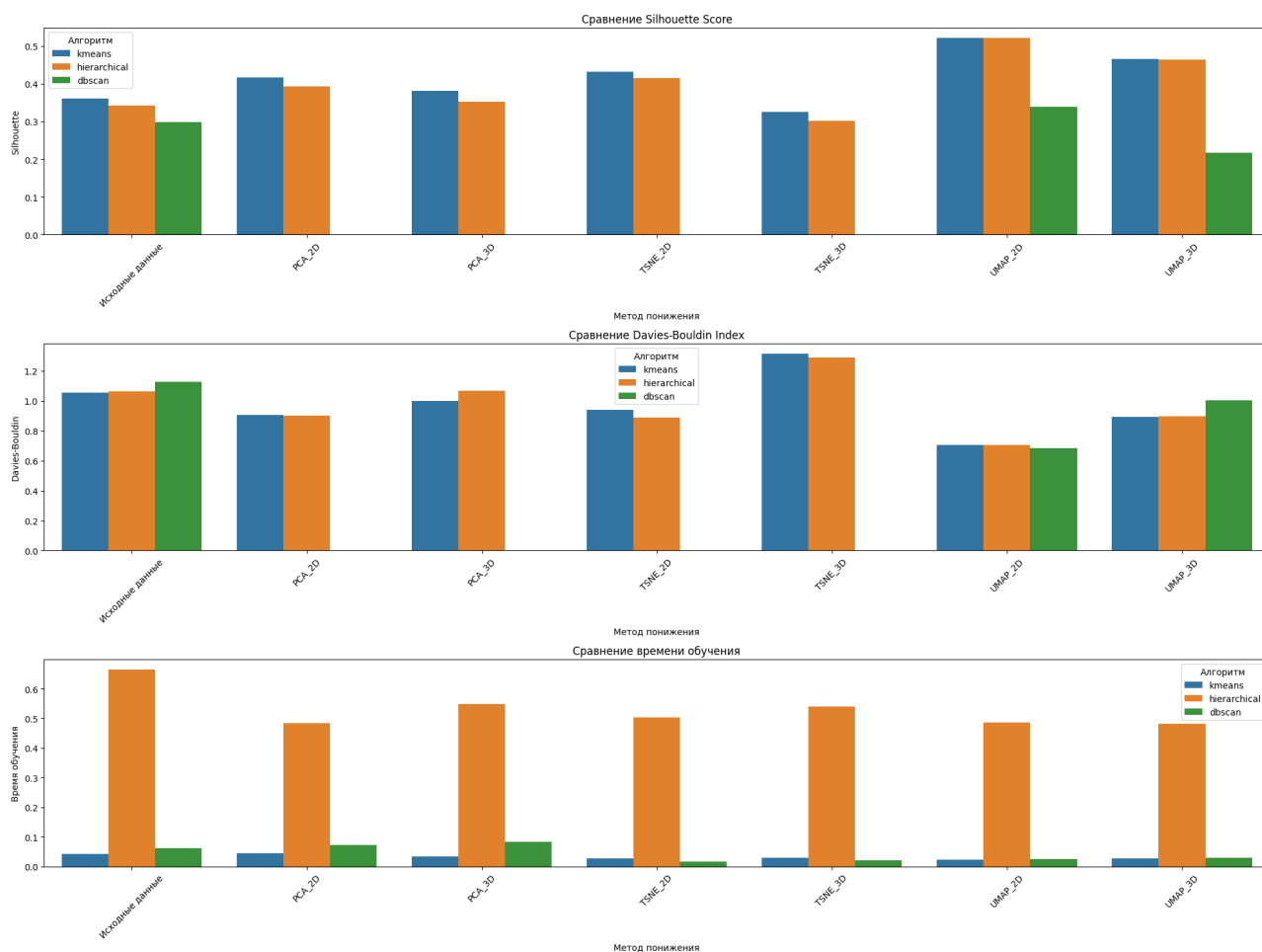


Рисунок 17 — Метрики каждого из методов

Сравнение метрик кластеризации до и после снижения размерности демонстрирует значительные различия в качестве результатов и эффективности алгоритмов. На исходных данных (без снижения размерности) K-means показал Silhouette Score 0.3610, Davies-Bouldin Index 1.0537 и Calinski-Harabasz Index 2462.42, что указывает на умеренное качество кластеризации с частичным пересечением кластеров. Иерархическая кластеризация (ward) имела схожие метрики (Silhouette 0.3427, Davies-Bouldin 1.0632), а DBSCAN выделил 630 шумовых точек с низким силуэтом (0.2981), что подчеркивало слабую применимость к исходным данным. После применения методов снижения размерности (PCA, t-SNE, UMAP) наблюдается явное улучшение метрик, особенно для нелинейных методов.

UMAP 2D стал наиболее эффективным преобразованием: для K-means и иерархической кластеризации Silhouette Score увеличился до 0.5218 (на 44%

выше, чем на исходных данных), а Davies-Bouldin Index снизился до 0.7050 (на 33% лучше), что свидетельствует о четком разделении кластеров без значительного пересечения. Calinski-Harabasz Index вырос до 5245.72, подтверждая компактность кластеров. DBSCAN на UMAP 2D также показал прогресс: количество кластеров увеличилось до 12, а Silhouette Score вырос до 0.3388 (на 13% выше исходного), при этом количество шумовых точек сократилось. Визуализация подтверждает, что UMAP 2D сохраняет локальные структуры данных, что критично для кластеризации.

t-SNE 2D улучшил метрики K-means (Silhouette 0.4324) и иерархической кластеризации (Silhouette 0.4155), но уступает UMAP. DBSCAN на t-SNE 2D не смог сформировать кластеры (0 кластеров), что связано с неоднородным распределением данных. Для PCA 2D метрики также выросли по сравнению с исходными данными (K-means: Silhouette 0.4160, Davies-Bouldin 0.9058), но эффект менее выражен, чем у UMAP, особенно в 3D-пространстве, где PCA показал ухудшение (Silhouette 0.3805 для K-means).

Анализ зависимости метрик от количества компонент (графики «по размерностям») выявил, что увеличение числа компонент в PCA приводит к снижению Silhouette Score и росту Davies-Bouldin Index, что подчеркивает избыточность высоких размерностей. Напротив, UMAP и t-SNE в 2D-формате обеспечивают оптимальный баланс между сохранением структуры данных и упрощением кластеризации. По времени обучения K-means остается лидером (0.015–0.025 сек на UMAP 2D), а иерархические методы, несмотря на большую вычислительную сложность, оправдывают затраты улучшенным качеством кластеров.

ВЫВОД

Проведенный анализ кластеризации на датасете "Online Retail" с применением различных методов кластеризации и снижения размерности показал, что методы снижения размерности, особенно UMAP 2D, значительно улучшают качество кластеризации по сравнению с исходными данными.

На исходных данных иерархические методы (Single и Average linkage) демонстрировали лучшее качество кластеризации (Silhouette Score 0.8518 и 0.8148 соответственно), однако общее качество оставалось умеренным, а DBSCAN выделял большое количество шумовых точек (630).

После применения методов снижения размерности наблюдается явное улучшение метрик: UMAP 2D стал наиболее эффективным преобразованием, обеспечив максимальные значения Silhouette Score (0.5218) и минимальный Davies-Bouldin Index (0.7050) для K-means и иерархической кластеризации, что подтверждает четкое разделение кластеров без значительных пересечений.

Визуализация данных на UMAP 2D подтверждает сохранение локальных структур данных, что критически важно для качественной кластеризации. Методы PCA и t-SNE также улучшили результаты, но уступают UMAP: PCA показал умеренные результаты (Silhouette Score 0.4160 для K-means), а t-SNE 2D — хорошие (Silhouette 0.4324), но не такие выразительные, как UMAP. DBSCAN демонстрирует нестабильность на исходных данных и после PCA/t-SNE (формирует 0 или 1 кластер), но на UMAP 2D успешно выделяет 12 кластеров с умеренным качеством (Silhouette Score 0.3388). По скорости обучения K-means остается лидером (0.015–0.025 сек на UMAP 2D), что делает его подходящим для задач с временными ограничениями, тогда как иерархические методы, несмотря на большую вычислительную сложность (0.3–0.6 сек), оправдывают затраты улучшенным качеством кластеров.

Анализ зависимости метрик от количества компонент выявил, что для PCA увеличение числа компонент приводит к ухудшению качества кластеризации,

тогда как UMAP и t-SNE в 2D-формате обеспечивают оптимальный баланс между сохранением структуры данных и упрощением кластеризации.

Таким образом, для задач кластеризации с подобными данными рекомендуется использовать UMAP 2D в сочетании с K-means или иерархическими методами, что обеспечивает наилучшее качество при разумных временных затратах, избегая проблем, связанных с шумом и сложной структурой исходных данных.